

Multi-task Inference of Diffusion Networks

Ting Gan
School of Computer Science,
Wuhan University
Wuhan, China
ganting@whu.edu.cn

Kudereti Kuerban
School of Cyber Science and
Engineering, Wuhan University
Wuhan, China
kudret@whu.edu.cn

Qian Yan
School of Computer Science,
Wuhan University
Wuhan, China
qy@whu.edu.cn

Ling Han
Department of Computer Science,
Yale University
New Haven, CT, USA
l.han@yale.edu

Zhigao Zheng*
School of Computer Science,
Wuhan University
Wuhan, China
zhengzhigao@whu.edu.cn

Hao Huang*
School of Computer Science,
Wuhan University
Wuhan, China
haohuang@whu.edu.cn

Abstract

Inferring the underlying structures of diffusion networks based on observed diffusion results is a fundamental problem in network analysis. Traditional approaches typically address this problem by inferring each diffusion network in isolation, relying on the assumption that sufficient observation data is available for each individual inference task. However, in many real-world scenarios, it is common to observe diffusion processes occurring across multiple networks with similar structures, while the amount of observable data collected on each network is often limited. In this work, we study how to infer multiple similar diffusion networks jointly with limited observation data for each network. To this end, we propose a novel iterative strategy which in turn updates the inference results for all diffusion networks by exploiting the similarity between the networks, and theoretically guarantees the monotonicity and convergence of the iterative process. Extensive experiments on both synthetic and real-world networks demonstrate that our method not only achieves superior inference accuracy compared to existing techniques, but also maintains high computational efficiency.

CCS Concepts

• **Computing methodologies** → **Maximum likelihood modeling**; • **Mathematics of computing** → **Computing most probable explanation**.

Keywords

Diffusion Network; Influence Relationship; Multi-task Inference

ACM Reference Format:

Ting Gan, Kudereti Kuerban, Qian Yan, Ling Han, Zhigao Zheng, and Hao Huang. 2026. Multi-task Inference of Diffusion Networks. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792222>

*Zhigao Zheng and Hao Huang are the co-corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792222>

1 Introduction

A diffusion network is usually modeled as a directed graph, where a directed edge from a parent node to a child node indicates that the parent node can influence the child node with a certain probability [3, 4, 16, 20]. The structure of a diffusion network delineates the underlying influence relationships between network nodes. Such a structure is essential for understanding the intrinsic diffusion mechanisms and developing strategies to control future diffusion on the network. Nonetheless, the underlying structures of diffusion networks are often not naturally accessible in practice, and need to be reconstructed through inference based on observation data collected from historical diffusion processes [2, 6, 14, 35].

To infer diffusion network structures, most existing approaches resort to precise timestamps of historical node infections (known as cascades) [4, 7–11, 19, 22–25, 27–29], since they assume that nodes infected sequentially within narrow time windows are more likely to possess influence relationships. In reality, tracing exact cascades is often difficult and expensive, especially in diffusion processes with long-term durations and wide node distributions, such as the spread of epidemics. Therefore, a few approaches try to infer diffusion network structures without cascades, using only the eventual infection statuses of nodes in historical diffusion processes [1, 12, 16–18], since observing the final infection statuses of nodes at the end of each diffusion process is more feasible and affordable than constantly monitoring the nodes to trace cascades.

Regardless of the type of observation data used, existing approaches infer diffusion network independently, and assume that sufficient observation data for each inference task is available, from which a reliable structure can be inferred. In many real-world situations, however, it is difficult to gather enough observation data on every diffusion network. Fortunately, observation data can be gathered on multiple closely related diffusion networks. Taking microblogging for example, at different periods, the follower-follower relationships among users often have a lot in common but not be identical. These relationships vary slightly over time, forming a set of diffusion networks with similar structures, and at each period, the amount of available observation data on each network is often limited. For this kind of situations, multi-task learning [36] suggests that it may be possible to achieve more accurate inference results by leveraging the relatedness across multiple related inference tasks. Nevertheless, so far, few studies have effectively leveraged the similarity across multiple

diffusion networks to infer more accurate diffusion network structures. Although several existing work can infer multiple diffusion networks simultaneously [15, 31], they still deal with each inference task independently, and can not take advantage of the similarity across diffusion networks.

In this work, we investigate how to infer multiple similar diffusion networks jointly with limited observation data for each network. We propose an effective and efficient algorithm called MIND (a re-ordered acronym of **M**ulti-task **I**nferece of **D**iffusion **N**etworks) for this problem. Equipped with a probabilistic generative model, MIND is able to estimate the likelihood of all inferred network structures jointly, by exploiting the similarity between the networks. In addition, MIND adopts an iterative strategy which in turn updates each inferred network structure in each iteration. It can be theoretically guaranteed that the likelihood of inferred network structures increases monotonically through the iterations of MIND, and the iteration process terminates in finite steps.

The remainder of the paper is organized as follows. Section 2 reviews related work, and Section 3 defines the problem. Section 4 details the proposed MIND algorithm, Section 5 reports experimental results, and Section 6 concludes the paper.

2 Related Work

2.1 Cascade-based Approaches

The majority of existing approaches assume that the exact timestamp of each node infection is available. With these infection timestamps as cascades, they infer influence relationships between nodes by identifying frequent sequences in cascades.

These cascade-based approaches often transform the problem of diffusion network inference into an optimization problem, by modeling the likelihood of cascades with a convex optimization objective function. Then, they find the optimal diffusion network structure, which is most likely to generate cascades, with diverse techniques, such as sequential quadratic programming [7, 23], EM algorithm [31], stochastic gradient methods [4, 10], convex risk minimization [30], survival theory [9], sparse recovery [27], neural mean-field dynamics [13], or decoupling into multiple parallelizable subproblems [19, 24]. Besides using convex optimization objective functions, several approaches adopt an objective function with submodular property [8, 11] and use greedy algorithms for solving the objective. NetInf [8] and MulTree [11] are representative approaches: NetInf prioritizes efficiency by focusing on the most probable propagation tree, while MulTree emphasizes accuracy by considering all cascade-supported trees.

Departing from these cascade-based approaches that customarily focus on inferring a single diffusion network, our MIND algorithm is designed for inferring multiple diffusion networks with similar structures, and is based on only the final infection statuses of nodes.

2.2 Infection Status-based Approaches

Since cascade tracing is often prohibitively expensive, several studies have explored inference approaches that rely solely on the final infection states of nodes in historical diffusion processes, which is more readily available in practical scenarios.

The infection status-based approaches investigate either the joint influence effects of the parent nodes set under different

combinations of infection statuses [6, 12, 15–17], or the independent influence effect of each parent node [14, 18, 32, 33], on the infection of a child node based on historical data of node infection statuses, and reconstruct diffusion networks based on bayesian inference or gradient descent techniques. Due to solid mathematical foundations, a few of these approaches, such as SIDN [16] and TENDS [12], provide a theoretical guarantee on their accuracy performance.

Most of infection status-based approaches aim to infer a single diffusion network. Only the LMDN algorithm [15] tries to infer a multi-aspect diffusion network, in which each aspect refers to a single diffusion network. LMDN assumes that different aspects have different structures, and infers each diffusion network separately. In contrast, our MIND algorithm assumes that the target multiple diffusion networks share similar structures, and leverages the structure similarities to improve inference accuracy.

3 Problem Statement

A diffusion network is formally modeled as a directed graph $G = (V, E)$, where V represents the set of nodes and E denotes the directed edges capturing potential propagation pathways. In the directed graph, a directed edge from a parent node to a child node represents an influence relationship between them, which indicates that when the parent node is infected and the child node is uninfected, the parent node will successfully infect the child node with a certain probability. Formally, suppose there are k diffusion networks with identical node set V , namely, $G_1, \dots, G_k$, directed graph $G_t = (V, E_t)$ ($t \in \{1, \dots, k\}$) denotes the t -th diffusion network, where $V = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes, and E_t is the set of directed edges in G_t .

Let $F_{t,i} \subset V$ denote the set of parent nodes for node $v_i \in V$ with respect to the edge set E_t . For each node $v_i \in V$, the infection status is binary: infected (denoted by 1) or uninfected (denoted by 0). Consequently, the set of parent nodes $F_{t,i}$ admits $2^{|F_{t,i}|}$ possible infection status configurations, where $|F_{t,i}|$ denotes the cardinality of $F_{t,i}$ (i.e., the number of nodes in set $F_{t,i}$). Let $X_i \in \{0, 1\}$ be the infection status variable of node v_i , and $X_{F_{t,i}} \in \{0, 1\}^{|F_{t,i}|}$ be the joint infection status variables of nodes in its parent set $F_{t,i}$. For each $j \in \{1, \dots, 2^{|F_{t,i}|}\}$, let $\pi_{t,i,j} \in \{0, 1\}^{|F_{t,i}|}$ be the j -th possible infection status configuration of $F_{t,i}$. We define $\theta_{t,i,j,1} = P(X_i = 1 \mid X_{F_{t,i}} = \pi_{t,i,j})$ as the conditional infection probability of v_i being infected under the condition that $F_{t,i}$ is instantiated to the j -th possible infection status configuration. For convenience, we define $\theta_{t,i,j,0} = 1 - \theta_{t,i,j,1} = P(X_i = 0 \mid X_{F_{t,i}} = \pi_{t,i,j})$.

In this paper, we focus on inferring multiple similar diffusion networks jointly with limited observation data for each network. In this problem, we assume that the k objective networks $G_1, \dots, G_k$ have similar structures, and observation data consists of only the eventual infection statuses of nodes from historical diffusion processes. The problem can be formally stated as follows.

Given: k data sets $D_1, \dots, D_k$ of infection status results observed on k diffusion networks $G_1, \dots, G_k$, where $D_t = \{S_t^1, \dots, S_t^{\beta_t}\}$ is the infection status results observed on $G_t = (V, E_t)$ in β_t diffusion processes ($t \in \{1, \dots, k\}$), and $S_t^\ell = \{s_{t,1}^\ell, \dots, s_{t,n}^\ell\} \in \{0, 1\}^n$ is a n -dimensional 0-1 vector that records the eventual infection status $s_{t,i}^\ell \in \{0, 1\}$ of $v_i \in V$ in the ℓ -th diffusion process ($\ell \in \{1, \dots, \beta_t\}$).

Infer: the edge set E_t of G_t for each $t \in \{1, \dots, k\}$.

4 The MIND Algorithm

In this section, we first explain how to measure the likelihood of inferred diffusion network structures jointly, followed by introducing how to update each inferred network structure in an iterative way. Then we present the detailed steps of the MIND algorithm, and conclude this section with a complexity analysis on the algorithm.

4.1 Measuring Likelihood of Inferred Structures

Given the infection status results $D_1, \dots, D_k$, the multi-task inference of diffusion networks is equivalent to finding k diffusion networks $G_1, \dots, G_k$ that maximize the following posterior probability:

$$G_1, \dots, G_k = \arg \max_{G_1, \dots, G_k} p(G_1, \dots, G_k \mid D_1, \dots, D_k). \quad (1)$$

According to Bayes' rule, we have

$$\begin{aligned} p(G_1, \dots, G_k \mid D_1, \dots, D_k) \\ = \frac{p(G_1, \dots, G_k) p(D_1, \dots, D_k \mid G_1, \dots, G_k)}{p(D_1, \dots, D_k)}. \end{aligned} \quad (2)$$

As each D_t ($t \in \{1, \dots, k\}$) is collected from corresponding G_t independently, we can reformulate $p(D_1, \dots, D_k \mid G_1, \dots, G_k)$ as

$$p(D_1, \dots, D_k \mid G_1, \dots, G_k) = \prod_{t=1}^k p(D_t \mid G_t). \quad (3)$$

As there is no prior knowledge on the probability distribution of $D_1, \dots, D_k$, we equally treat each possible value of $p(D_1, \dots, D_k)$ by assuming that $p(D_1, \dots, D_k)$ is equal to a certain constant. Then, the problem in Eq. (1) is equivalent to the following problem.

$$G_1, \dots, G_k = \arg \max_{G_1, \dots, G_k} \mathcal{L}(G_1, \dots, G_k) \quad (4)$$

where $\mathcal{L}(G_1, \dots, G_k) = p(G_1, \dots, G_k) \prod_{t=1}^k p(D_t \mid G_t)$.

As it is difficult and expensive to find the globally optimal $G_1, \dots, G_k$, we adopt a local search method. To be specific, during the search of the structure of G_t ($t \in \{1, \dots, k\}$), we fix the structures of the other networks with their current inference results. With this method, we infer the structure of each network in turn. In this situation, $\mathcal{L}(G_1, \dots, G_k)$ can be reformulated as follows.

$$\mathcal{L}(G_1, \dots, G_k) = p(D_t \mid G_t) p(G_t \mid G_{-t}) Q(G_{-t}), \quad (5)$$

where $G_{-t} = \{G_1, \dots, G_{t-1}, G_{t+1}, \dots, G_k\}$ refers to the remaining networks after removing G_t from $\{G_1, \dots, G_k\}$, and $Q(G_{-t}) = p(G_{-t}) \prod_{s \neq t} p(D_s \mid G_s)$.

The three parts in $\mathcal{L}(G_1, \dots, G_k)$ work differently:

(1) Probability $p(D_t \mid G_t)$ measures the likelihood that the infection status result D_t is generated by G_t . According to Eq. (3) in TENDS [12], $p(D_t \mid G_t)$ can be calculated as:

$$p(D_t \mid G_t) = \prod_{i=1}^n \prod_{j=1}^{2^{F_{t,i}}} \prod_{r=0}^1 \theta_{t,i,j,r}^{N_{t,i,j,r}}. \quad (6)$$

where $\theta_{t,i,j,1}$ is the conditional infection probability of v_i being infected under the condition that $F_{t,i}$ is instantiated to the j -th possible infection status configuration, $\theta_{t,i,j,0} = 1 - \theta_{t,i,j,1}$, and $N_{t,i,j,r}$ is the number of times situations $X_i = r \wedge X_{F_{t,i}} = \pi_{t,i,j}$ appear in D_t .

(2) Probability $p(G_t \mid G_{-t})$ encodes how similar network G_t and the other networks should be. Due to the similarity across the

networks, if an edge frequently appears in the other networks, the probability of this edge existing in G_t is high. Let $e_{t,i,j} \in \{0, 1\}$ indicates whether there is an influence relationship from v_i to v_j in G_t (1 for yes and 0 for no), the probability that directed edge (v_i, v_j) exists in G_t can be estimated as follows.

$$p(e_{t,i,j} = 1 \mid G_{-t}) = \frac{\sum_{s \neq t} e_{s,i,j}}{k-1}, \quad (7)$$

A slight limitation manifests itself in the fact that the estimated probability could be 0 or 1, when an edge exists in none or all of the other networks. In these two situations, the estimated probability for this edge will be fixed at 0 or 1 without any change, and thus fails to help update the inference result of the corresponding edge in G_t . To help the probability estimated by Eq. (7) fall in $(0, 1)$, we introduce a perturbation factor α in the numerator and a perturbation factor $\tau\alpha$ in the denominator. Subsequently, the estimated probability can be reformulated as follows.

$$p(e_{t,i,j} = 1 \mid G_{-t}) = \frac{\sum_{s \neq t} e_{s,i,j} + \alpha}{k-1 + \tau\alpha}, \quad (8)$$

where α is a small value (α is set to 0.0001 in this paper), and $\tau > 1$ is a hyperparameter (τ is set to 2 in this paper). These perturbation factors can effectively prevent $p(e_{t,i,j} \mid G_{-t})$ from being 0 or 1, thus facilitating the exploration of similarity across the networks. As $p(e_{t,i,j} = 0 \mid G_{-t}) = 1 - p(e_{t,i,j} = 1 \mid G_{-t})$, probability $p(G_t \mid G_{-t})$ can be calculated as follows.

$$p(G_t \mid G_{-t}) = \prod_{i=1}^n \prod_{v_j \in F_{t,i}} p_{t,j,i} \prod_{v_j \notin F_{t,i} \cup \{v_i\}} (1 - p_{t,j,i}), \quad (9)$$

where $p_{t,j,i} = p(e_{t,j,i} = 1 \mid G_{-t})$.

(3) $Q(G_{-t})$ is independent w.r.t. G_t .

We can iteratively perform the local search method to approximate the optimal solution. Before the first iteration, we initialize the inference result for the structure of each network G_t ($t \in \{1, \dots, k\}$) by executing an existing infection status-based approach SIDN [16] with infection status result D_t . Let $G_1^{(T)}, \dots, G_k^{(T)}$ be the inferred networks after T iterations. In $(T+1)$ -th iteration, when the first $t-1$ networks have been updated as $G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}$, what we need to do is finding a $G_t^{(T+1)}$ that maximizes the value of the following objective function.

$$G_t^{(T+1)} = \arg \max_{G_t} f_t^{(T+1)}(G_t), \quad (10)$$

where $f_t^{(T+1)}(G_t)$ represents the formula below:

$$p(D_t \mid G_t) p(G_t \mid G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}).$$

We would like to point out that the $G_t^{(T+1)}$ found by Eq. (10) tends to increase the likelihood of inferred diffusion network structures. This nice property can be explained by the following Lemma 1. Note that all the proofs related to this section are given in the appendix.

LEMMA 1. If $G_t^{(T+1)}$ is the solution obtained by solving the optimization problem in Eq. (10), then

$$\begin{aligned} \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}) \\ \leq \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T+1)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}). \end{aligned} \quad (11)$$

Combining Eq. (6) and Eq. (9) into $f_t^{(T+1)}(G_t)$, we have

$$\begin{aligned} & \ln f_t^{(T+1)}(G_t) \\ &= \sum_{i=1}^n \sum_{j=1}^{2^{F_{t,i}}} \sum_{r=0}^1 N_{t,i,j,r} \ln \theta_{t,i,j,r} + \sum_{i=1}^n \sum_{v_j \in F_{t,i}} \ln \frac{\sum_{s \neq t} \hat{e}_{s,j,i} + \alpha}{k-1+2\alpha} \\ &+ \sum_{i=1}^n \sum_{v_j \notin F_{t,i} \cup \{v_i\}} \ln \frac{k-1-\sum_{s \neq t} \hat{e}_{s,j,i} + \alpha}{k-1+2\alpha} \end{aligned} \quad (12)$$

where $\hat{e}_{s,j,i} = e_{s,j,i}^{(T+1)}$ for $s \in \{1, \dots, t-1\}$, and $\hat{e}_{s,j,i} = e_{s,j,i}^{(T)}$ for $s \in \{t+1, \dots, k\}$. A greater value of $\ln f_t^{(T+1)}(G_t)$ indicates a higher likelihood of the inferred structure of G_t . Here, we obtain the final optimization objective as Eq. (12), which could help us find the optimal $G_t^{(T+1)}$ in the $(T+1)$ -th iteration.

4.2 Updating Inferred Structures

To maximize the value of $\ln f_t^{(T+1)}(G_t)$, we should find for each node v_i an optimal parent node set $F_{t,i} \subseteq V \setminus \{v_i\}$ that maximizes the following scoring function.

$$\begin{aligned} & g_t^{(T+1)}(v_i, F_{t,i}) \\ &= \sum_{j=1}^{2^{F_{t,i}}} \sum_{r=0}^1 N_{t,i,j,r} \ln \frac{N_{t,i,j,r}}{N_{t,i,j}} + \sum_{v_j \in F_{t,i}} \ln \frac{\sum_{s \neq t} \hat{e}_{s,j,i} + \alpha}{k-1+2\alpha} \\ &+ \sum_{v_j \notin F_{t,i} \cup \{v_i\}} \ln \frac{k-1-\sum_{s \neq t} \hat{e}_{s,j,i} + \alpha}{k-1+2\alpha}. \end{aligned} \quad (13)$$

Then, the structure inference task for G_t is decomposed into finding each node a set of optimal parent nodes:

$$\max_{F_{t,i}} g_t^{(T+1)}(v_i, F_{t,i}). \quad (14)$$

This is a discrete optimization problem. As there are 2^{n-1} possible solutions for this problem, it is too expensive to enumerate all of them. Therefore, we adjust the optimization problem in Eq. (14) as a constrained optimization problem:

$$\begin{aligned} & \max_{F_{t,i}} g_t^{(T+1)}(v_i, F_{t,i}) \\ & \text{s.t. } |F_{t,i} \setminus F_{t,i}^{(T)}| + |F_{t,i}^{(T)} \setminus F_{t,i}| \leq 1, \end{aligned} \quad (15)$$

where $|A \setminus B|$ represents the number of elements in A but not in B . This constraint specifies that in the $(T+1)$ -th iteration of local search method, the $F_{t,i}$ should be updated by removing at most one node from, or adding at most one new node into, v_i 's parent node set $F_{t,i}^{(T)}$ in T -th iteration.

Based on the scoring function in Eq. (13) and parent node set updating method specified by the constraint in Eq. (15), we propose the updating algorithm for G_t in Algorithm 1.

Algorithm 1 takes as inputs the infection status result D_t observed on G_t , and current inferred networks $G_1, \dots, G_k$. Its objective is to update the parent node set $F_{t,i}$ for each node v_i . To facilitate this process, the algorithm introduces two auxiliary variables, $F'_{t,i}$ and g_m , to record the searched optimal parent node set and its corresponding score, respectively. Initially, $F'_{t,i}$ is assigned the value of $F_{t,i}$, and g_m is set to the corresponding

Algorithm 1 Updating algorithm for G_t

Input: $D_t, G_1, \dots, G_k$.

Output: G_t .

```

1: for  $v_i \in V$  do
2:    $g_m = g_t^{(T+1)}(v_i, F_{t,i}), F'_{t,i} = F_{t,i}$ ;
3:   for  $v_j \in V \setminus \{v_i\}$  do
4:     if  $v_j \in F_{t,i}$  then
5:        $F_{new} = F_{t,i} \setminus \{v_j\}$ ;
6:     else
7:        $F_{new} = F_{t,i} \cup \{v_j\}$ ;
8:     end if
9:     if  $g_t^{(T+1)}(v_i, F_{new}) > g_m$  then
10:       $g_m = g_t^{(T+1)}(v_i, F_{new}), F'_{t,i} = F_{new}$ ;
11:    end if
12:  end for
13: end for
14:  $E_t = \{(v_j, v_i) \mid v_i \in V, v_j \in F'_{t,i}\}$ ;
15: return  $G_t = (V, E_t)$ .
```

score $g_t^{(T+1)}(v_i, F_{t,i})$ (Line 2). The algorithm proceeds by iterating over each node $v_j \in V \setminus \{v_i\}$. If v_j already exists in $F_{t,i}$ (Line 4), a new parent node set $F_{new} = F_{t,i} \setminus \{v_j\}$ is generated by removing v_j from $F_{t,i}$ (Line 5). Conversely, if v_j does not exist in $F_{t,i}$ (Line 6), a new parent node set $F_{new} = F_{t,i} \cup \{v_j\}$ is generated by adding v_j to $F_{t,i}$ (Line 7). Then, the score $g_t^{(T+1)}(v_i, F_{new})$ corresponding to F_{new} is calculated. If this score is higher than the current optimal value g_m (Line 9), the algorithm updates g_m to the new score $g_t^{(T+1)}(v_i, F_{new})$, and sets $F'_{t,i} = F_{new}$ (Line 10). After all candidate nodes have been examined, the updated parent node set $F'_{t,i}$ is obtained. Finally, the algorithm updates the edge set E_t of G_t based on the updated parent node sets (line 14), and return the updated G_t (line 15).

In the $(T+1)$ -th iteration of local search method, the $G_t^{(T+1)}$ updated by Algorithm 1 tends to increase the likelihood of inferred diffusion network structures. This nice property can be explained by the following Lemma 2.

LEMMA 2. Assume that the input of Algorithm 1 are $D_t, G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}$ and the output is $G_t^{(T+1)}$, then

$$\begin{aligned} & \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}) \\ & \leq \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T+1)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}), \end{aligned} \quad (16)$$

where the equal sign holds iff $G_t^{(T+1)} = G_t^{(T)}$.

4.3 Algorithm

Based on the updating algorithm for G_t , we propose the MIND algorithm to infer multiple similar diffusion networks jointly.

The MIND algorithm, outlined in Algorithm 2, takes as inputs the infection status results $D_1, \dots, D_k$. It first uses SIDN to initialize the structure of each network G_t ($t \in \{1, \dots, k\}$) (line 1), then updates networks $G_1, \dots, G_k$ by Algorithm 1 in turn and checks whether there are changes in the updated results (lines 3–12). If updated results no longer change, they are returned by MIND as the final inferred networks $G_1, \dots, G_k$ (line 13).

Algorithm 2 The MIND algorithm

Input: $D_1, \dots, D_k$.
Output: $G_1, \dots, G_k$.

```

1: Use SIDN to initialize  $G_t$  with  $D_t, \forall t \in \{1, \dots, k\}$ ;
2:  $change = true$ ;
3: while  $change = true$  do
4:    $change = false$ ;
5:   for  $t \in \{1, \dots, k\}$  do
6:     Using Algorithm 1 to update  $G_t$  as  $G'_t$ ;
7:     if  $G'_t \neq G_t$  then
8:        $change = true$ ;
9:        $G_t = G'_t$ ;
10:    end if
11:  end for
12: end while
13: return  $G_1, \dots, G_k$ .

```

The MIND algorithm adopts the local search method, and infers $G_1, \dots, G_k$ in an iterative way. The following Theorem 1 can theoretically guarantee that with the increasing of iteration times, the likelihood of diffusion network structures inferred by MIND algorithm will increase monotonically.

THEOREM 1. Assume that $G_1^{(T)}, \dots, G_k^{(T)}$ are the networks inferred by MIND in the T -th iteration, and $G_1^{(T+1)}, \dots, G_k^{(T+1)}$ are the networks inferred by MIND in the $(T+1)$ -th iteration, then

$$\mathcal{L}(G_1^{(T)}, \dots, G_k^{(T)}) \leq \mathcal{L}(G_1^{(T+1)}, \dots, G_k^{(T+1)}), \quad (17)$$

where the equal sign holds iff $G_t^{(T+1)} = G_t^{(T)}, \forall 1 \leq t \leq k$.

Furthermore, the following Theorem 2 theoretically guarantees the convergence of MIND algorithm.

THEOREM 2. The MIND algorithm terminates in finite steps.

4.4 Complexity Analysis

In the updating algorithm for G_t (i.e., Algorithm 1), the most computationally expensive process is calculating the scoring function $g_t^{(T+1)}(v_i, F_{t,i})$ for each v_i and a possible parent node set $F_{t,i}$. The time complexity of calculating $g_t^{(T+1)}(v_i, F_{t,i})$ is about $O(\beta_t 2^\eta + k\eta)$, where β_t is the number of historical diffusion processes in D_t , k is the number of networks, and η is the upper bound of the parent node set size. Since η is determined by SIDN, the value is about $\log \beta_t$ and is much less than the number n of network nodes. Due to the constraint in Eq. (15), the possible parent node sets for each v_i can be enumerated, and the amount is about $n - 1$. Therefore, the time complexity of updating algorithm for G_t is about $O(\beta_t 2^\eta n^2 + k\eta n^2)$.

In MIND algorithm, using SIDN to initialize each G_t ($t \in \{1, \dots, k\}$) takes $O(\beta_t n^2 + \eta^2 m^\eta n \beta_t)$ time, $m \ll n$ is the number of candidate parent nodes for each node pruned by SIDN. In the while loop, the most computationally expensive process is using Algorithm 1 to obtain $G'_1, \dots, G'_k$, which takes $O(k\beta 2^\eta n^2 + k^2 \eta n^2)$ time, where $\beta = \max\{\beta_1, \dots, \beta_k\}$. Therefore, the overall complexity of MIND is about $O(k\beta n^2 + k\eta^2 m^\eta n \beta + k\beta 2^\eta n^2 T + k^2 \eta n^2 T)$, where T is the number of iterations in MIND.

Table 1: Properties of synthetic networks

Graphs	Number n of Nodes	Average Degree
LFR1-5	2000, 4000, 6000, 8000, 10000	6
ER1-5	2000, 4000, 6000, 8000, 10000	6

Table 2: Properties of generated similar networks

Similar Networks	Original Network	Similarity
G1-5, G6-10, G11-17, G18-22, G23-27	LFR3	0.6, 0.65, 0.7, 0.75, 0.8
G28-32, G33-37, G38-42, G43-47	LFR1, LFR2, LFR4, LFR5	0.7
G'1-5, G'6-10, G'11-17, G'18-22, G'23-27	ER3	0.6, 0.65, 0.7, 0.75, 0.8
G'28-32, G'33-37, G'38-42, G'43-47	ER1, ER2, ER4, ER5	0.7

5 Experimental Evaluation

We first describe our experimental setup in this section, then evaluate our MIND algorithm on both synthetic and real-world networks. We investigate the effects of the number of networks, similarity between networks, network size, initial infection ratio, the amount of diffusion processes, and the iterations of MIND, on the performance of MIND. All algorithms were implemented in Python and executed on a desktop PC equipped with an Intel Core i9-9900K processor (3.60 GHz) and 32GB of RAM. The source code of MIND and data sets used in the experiments are publicly available at <https://github.com/DiffusionNetwork/MIND>.

5.1 Experimental Setup

Network. We adopt LFR benchmark graphs [21] and ER random networks [5] as the synthetic networks. We generate multiple LFR benchmark graphs and ER random networks by setting different parameters, including the number of nodes and average node degree, with their properties detailed in Table 1. To generate multiple networks with similar structures for multi-task inference, we utilize LFR and ER networks as original networks, then randomly remove a few directed edges from original networks. Table 2 summarizes the properties of the generated similar networks. The similarity between two networks G and G' is measured by vertex/edge overlap [26], which is calculated as $similarity(G, G') = 2 \times \frac{|V \cap V'| + |E \cap E'|}{|V| + |V'| + |E| + |E'|}$. Furthermore, we construct two groups of similar networks based on two real-world microblogging networks drawn from [31]: (1) the DUNF network comprising 750 users with 2,974 directed edges representing follower relationships, and (2) the DPU network comprising 1038 users with 11,385 directed edges representing follower relationships. For each real-world network, we randomly remove 1/3 of the edges to create five similar networks for testing. These two groups of similar diffusion networks are referred to as 5-DUNF and 5-DPU, respectively. Similar methods for generating multiple objective diffusion networks are commonly used in existing related studies [15, 31].

Infection Status Data. For each $t \in \{1, \dots, k\}$, we conduct β_k times of independent diffusion simulations on the network G_k to

generate the infection status observations D_t , with each simulation initialized by randomly selected seed nodes. In each simulation of the diffusion process, every infected node attempts to infect its uninfected child nodes with a transmission probability subject to a Gaussian distribution with a mean of 0.3 and a standard deviation of 0.05. In addition to binary infection status results, the experimental framework captures temporal cascade data (containing infection timestamps) for cascade-based algorithms. Similar generation approaches for binary infection status data and cascade data are well-documented in existing literature [1, 6, 8, 12, 14–17, 31, 34].

Performance Criterion. We measure the accuracy of structure inference results by the F-score (i.e., the harmonic mean of precision and recall) of the inferred directed edges, which can be calculated as $F\text{-score} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$, where $\text{precision} = \frac{N_{TP}}{N_{TP} + N_{FP}}$, $\text{recall} = \frac{N_{TP}}{N_{TP} + N_{FN}}$. N_{TP} is the number of true positives, i.e., the edges in the real network that are inferred correctly by the algorithm, N_{FP} is the number of false positives, i.e., edges that do not exist in the real network, but that are inferred falsely by the algorithm, and N_{FN} is the number of false negatives, i.e., edges that exist in the real network, but that are not inferred by the algorithm. We report the average F-score on all objective diffusion networks as the accuracy performance of each tested algorithm.

Benchmark Algorithms. We compare MIND against two existing approaches: MulTree [11], a classical cascade-based approach, and SIDN [16], a high-performance approach based on infection status. Since MulTree requires the number of edges as input, we provide it with the true number of edges in the network. As MulTree and SIDN infer each objective network independently, we also run MulTree and SIDN with a union strategy (referred to as MulTree Union and SIDN Union), in which infection data from all objective networks are concatenated into a single dataset to infer one network. In addition, we compare MIND with LMDN [15], which is a state-of-the-art algorithm for multi-aspect diffusion network inference. For LMDN, we regard each objective network as one aspect of a multi-aspect diffusion network. Furthermore, to verify the effectiveness of MIND in terms of exploiting the similarity across the networks, we execute a naive version of MIND (denoted as MIND Naive), which does not utilize any knowledge from other inferred networks by setting $\hat{e}_{s,j,i} = 0, \forall s, i, j$, in Eq. (13).

5.2 Effect of Number of Networks

To investigate the effect of the number of networks, we test the algorithms on five groups of LFR synthetic networks (i.e., G11-13, G11-14, G11-15, G11-16, G11-17) and five groups of ER synthetic networks (i.e., G'11-13, G'11-14, G'11-15, G'11-16, G'11-17), where the number of networks varies from 3 to 7. For each network, we simulate 300 diffusion processes (i.e., $\beta = 300$). In each simulation, we randomly select 15% of nodes as initially infected nodes.

Fig. 1 reports the accuracy performance (with standard deviation indicated by shaded area) and running time for each algorithm. The results reveal several observations: (1) MIND has the highest accuracy, and tends to achieve slightly higher accuracy when there are more networks for multi-task inference. This is because that MIND can effectively leverage the similarity across networks, and more networks bring more references for MIND to conduct multi-task inference. (2) The increasing number of networks has little

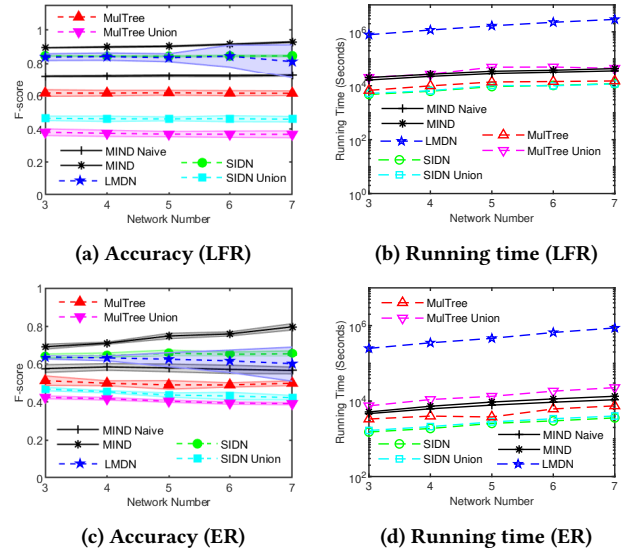


Figure 1: Effect of the number of networks

effect on the accuracy of other tested algorithms. This is because that these algorithms carry out inference tasks in isolation, and thus can not take advantage of the similarity across networks. Furthermore, the low accuracy of MulTree Union and SIDN Union indicates that the union strategy is not a good idea for inferring multiple similar networks. (3) More networks degrade the robustness of LMDN. Because for LMDN, more networks increase the risk of estimating wrong aspect labels for infection status results. When LMDN uses wrong infection status results to infer network structures, performance degradation happens. (4) MIND is much faster than LMDN, and shows competitive efficiency performance, compared with other tested algorithms. The running time of all algorithms exhibits a positive correlation with the number of networks.

5.3 Effect of Similarity between Networks

To study the effect of the similarity between networks, we test the algorithms on five groups of LFR synthetic networks (i.e., G1-5, G6-10, G11-15, G18-22, G23-27) and five groups of ER synthetic networks (i.e., G'1-5, G'6-10, G'11-15, G'18-22, G'23-27), with similarities varying from 0.6 to 0.8. A similarity threshold of 0.6 or higher is chosen for the following reason: since the chance that an edge between two nodes in one random network matches its presence or absence in another is 0.5, two random networks (same nodes) have an expected similarity around 0.5. When similarity < 0.5 , it is reasonable to assume that there is little to no positive correlation, or meaningful structural similarity, between the networks. In such cases, existing multi-aspect methods [15] that ignore inter-network similarity suffice. For each network, we simulate 300 diffusion processes (i.e., $\beta = 300$). In each simulation, we randomly select 15% of nodes as initially infected nodes.

Fig. 2 reports the performance of each algorithm, revealing several observations as follows: (1) MIND generally achieves the best accuracy, with SIDN and LMDN slightly outperforming it only when the LFR-network similarity is 0.6. This is because MIND

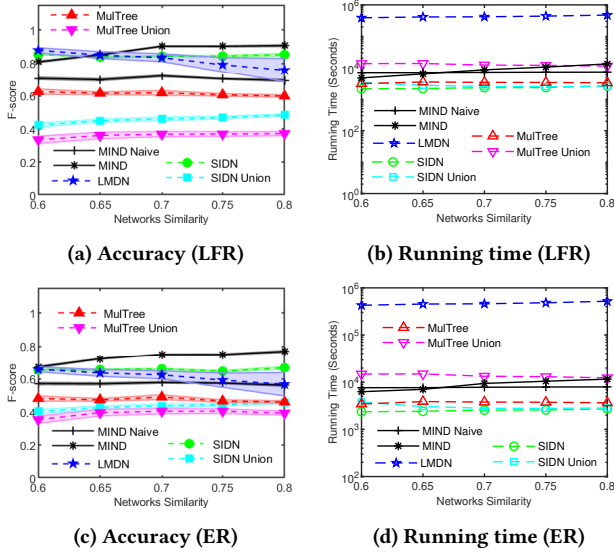


Figure 2: Effect of diffusion network similarity

leverages the similarity between networks to improve its inference results. A small similarity will restrict the above effect. Furthermore, for each group of inferred networks, we calculate the average similarity between networks returned by MIND, which turns out to be 0.39, 0.56, 0.64, 0.67, and 0.70 for LFR synthetic networks, and 0.36, 0.52, 0.60, 0.65, and 0.68 for ER synthetic networks, respectively. This indicates that although MIND exploits similarity between networks, it does not simply return similar structures but also preserves the differences between networks. (2) The accuracy of SIDN, MulTree, and MIND Naive is insensitive to the similarity between networks. Because they infer networks independently. (3) A greater similarity between networks improves the accuracy of MulTree Union and SIDN Union. This is because that these two algorithms infer only one network. If the objective networks are similar to each other, the difference between the objective networks and the inferred network tends to be reduced, which helps the two algorithms achieve a higher accuracy unintentionally. (4) A greater similarity between networks degrades the accuracy and robustness of LMDN. Because for LMDN, networks with more similar structures tends to increase the risk of estimating wrong aspect labels for infection status results, causing more wrong inference results. (5) MIND is much faster than LMDN, and shows competitive efficiency performance, compared with other tested algorithms. An increasing similarity between networks results in more running time for MIND and LMDN, and has little effect on the running time of other tested algorithms.

5.4 Effect of Network Size

To study the effect of diffusion network size, we test the algorithms across five groups of LFR benchmark networks (i.e., G28-G32, G33-G37, G11-G15, G38-G42, G43-G47) and five groups of ER synthetic networks (i.e., G'28-G32, G'33-G37, G'11-G15, G'38-G42, G'43-G47), with network sizes varying from 2000 to 10000. For each network, we simulate 300 diffusion processes (i.e., $\beta = 300$). In each simulation, we randomly select 15% of nodes as initially infected nodes.

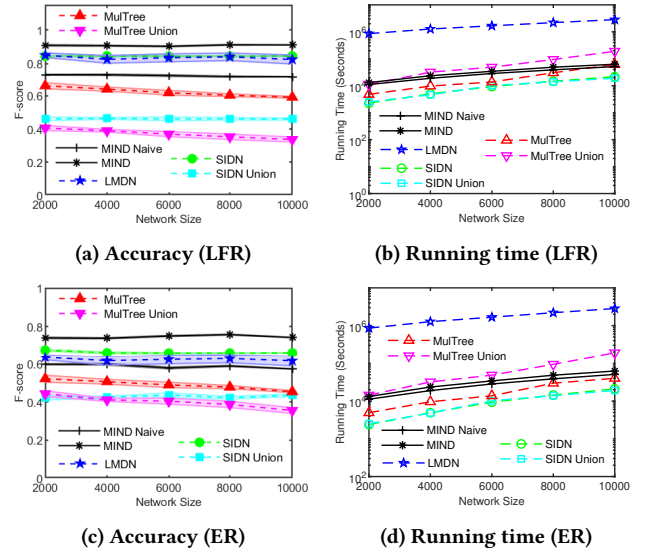


Figure 3: Effect of diffusion network size

Fig. 3 reports algorithms' performance, which demonstrates MIND's superior accuracy over other ones and reveals the following observations: (1) Increasing network size adversely affects the accuracy of MulTree and MulTree Union, while showing negligible impact on other tested algorithms. This phenomenon occurs because MulTree's exhaustive path enumeration strategy leads to combinatorial explosion in larger networks, i.e., the number of potential propagation paths grows exponentially with network size, which will increase the difficulty of the inference work of MulTree. MulTree Union shares the same reason. (2) All algorithms exhibit increasing running time with larger diffusion networks, with MIND demonstrating superior runtime efficiency compared to LMDN.

5.5 Effect of Initial Infection Ratio

To evaluate the effect of initial infection ratio, we test the algorithms on two groups of networks, i.e., 5-DUNF and 5-DPU, by varying the initial infection ratio from 0.05 to 0.25 with an interval of 0.05. For each network, we simulate 150 diffusion processes (i.e., $\beta = 150$).

Fig. 4 reports algorithms' performance, which demonstrates MIND's superior accuracy over other ones and reveals the following observations: (1) The accuracy of MIND is reasonably insensitive to the initial infection ratio. In contrast, the initial infection ratio shows significant effect on the benchmark algorithms. This is because the final infected nodes caused by a small fraction of initially infected nodes fail to comprehensively span all nodes within a target network. Consequently, the available infection data proves insufficient for benchmark algorithms to reveal all influence relationships among the nodes. Nonetheless, a large initial infection ratio can easily lead to false statistical correlations of node infections, which may reduce the accuracy of the algorithms. While MIND can relax this effect by leveraging the infection data across multiple similar networks. (2) An increase in the initial infection ratio may result in longer running time for MulTree and MulTree Union, and has little effect on other algorithms.

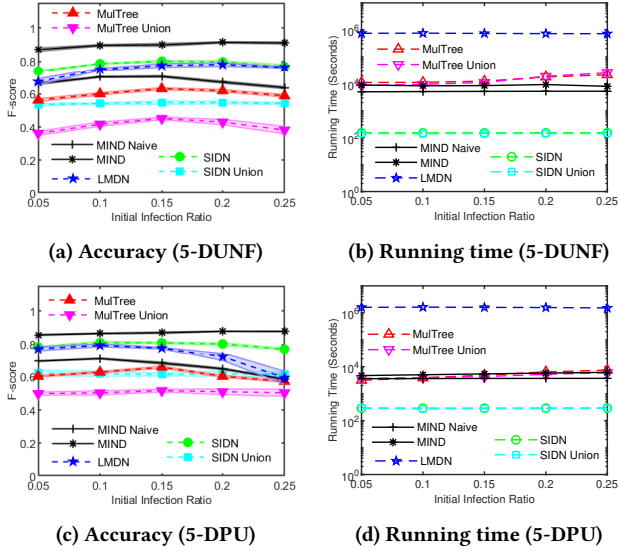


Figure 4: Effect of initial infection ratio

5.6 Effect of Diffusion Process Amount

To study the effect of diffusion process amount, we test the algorithms on two groups of networks, i.e., 5-DUNF and 5-DPU, with different amounts of diffusion processes. For each group of networks, we simulate different number β of diffusion processes (β varies from 100 to 200) on its five networks. In each simulation, we randomly select 15% of nodes as initially infected nodes.

Fig. 5 reports algorithms' performance, which demonstrates that MIND has significantly higher accuracy even with a small amount of observation data (i.e., a small amount of diffusion processes), and also reveals the following observations: (1) A greater amount of diffusion processes often helps the algorithms achieve more accurate results. The reason behind this is that data from more diffusion processes helps bring more information about objective networks, and helps the algorithms to have more accurate inference results. (2) To deal with the data collected from more diffusion processes, each algorithm often requires more running time.

5.7 Effect of Iterations of MIND

To evaluate the effect of MIND's iterations on its performance, we test MIND on the 5-DUNF and 5-DPU networks. For each network, we simulate 150 diffusion processes (i.e., $\beta = 150$), initializing each process with a random 15% of nodes ($0.15n$) as infected seeds. We vary the maximum number of iterations in MIND from 1 to 7, and report MIND's accuracy as well as the accumulated running time at the end of each iteration.

Fig. 6 illustrates the results of the test on the number of iterations, revealing two observations: (1) MIND exhibits monotonic accuracy improvement with more iterations, and demonstrates a reasonably fast convergence property; (2) the running time for MIND increases with more iterations as a rule.

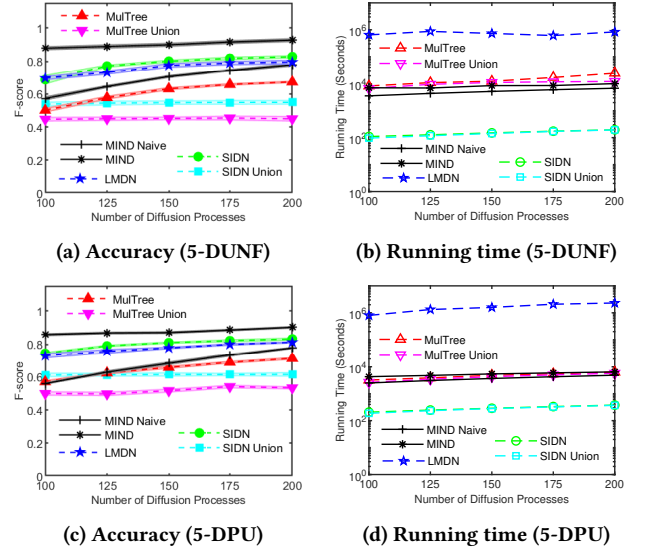


Figure 5: Effect of the amount of diffusion processes

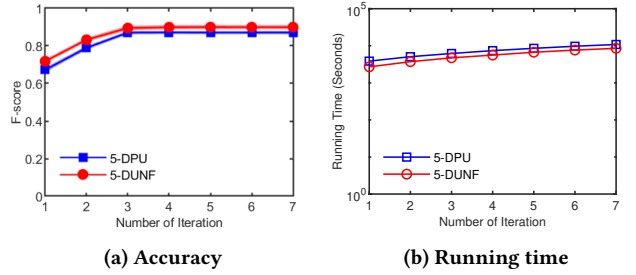


Figure 6: Effect of iterations of MIND

6 Conclusion

We investigated the problem of how to jointly infer multiple structurally similar diffusion networks with limited observation data for each individual network, and proposed MIND to solve this problem. MIND updates inferred network structures iteratively and jointly by exploiting the similarity between the networks. It can be theoretically guaranteed that the likelihood of inferred network structures increase monotonically through the iterations of MIND, and the iteration process terminates in finite steps. Through experimental results on both synthetic and real-world networks, the effectiveness and efficiency of MIND were verified.

Acknowledgments

This work was supported in part by the Science and Technology Program of XPCC under Grant 2025AA017, the Key Research and Development Program of Hubei Province under Grant 2024BAB044 and Grant 2023BAB170, and the National Natural Science Foundation of China under Grant 62372333 and Grant 62472328.

References

- [1] K. Amin, H. Heidari, and M. Kearns. 2014. Learning from Contagion(Without Timestamps). In *Proceedings of the 31st International Conference on Machine Learning (ICML 2014)*. 1845–1853.
- [2] S. Bourigault, C. Lagnier, S. Lamprier, L. Denoyer, and P. Gallinari. 2014. Learning social network embeddings for predicting information diffusion. In *Proceedings of the 7th ACM International Conference on Web Search and Data Mining (WSDM 2014)*. 393–402.
- [3] F. Croitoru, V. Hondru, R. Ionescu, and M. Shah. 2023. Diffusion models in vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 9 (2023), 10850–10869.
- [4] H. Daneshmand, M. Gomez-Rodriguez, L. Song, and B. Schölkopf. 2014. Estimating Diffusion Network Structures: Recovery Conditions, Sample Complexity & Soft-thresholding Algorithm. In *Proceedings of the 31st International Conference on Machine Learning (ICML 2014)*. 793–801.
- [5] P. Erdős and A. Rényi. 1959. On random graphs I. *Publicationes Mathematicae Debrecen* 6 (1959), 290–297.
- [6] T. Gan, K. Han, H. Huang, S. Ying, Y. Gao, and Z. Li. 2021. Diffusion Network Inference from Partial Observations. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI 2021)*. 7493–7500.
- [7] M. Gomez-Rodriguez, D. Balduzzi, and B. Schölkopf. 2011. Uncovering the Temporal Dynamics of Diffusion Networks. In *Proceedings of the 28th International Conference on Machine Learning (ICML 2011)*. 561–568.
- [8] M. Gomez-Rodriguez, J. Leskovec, and A. Krause. 2010. Inferring Networks of Diffusion and Influence. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2010)*. 1019–1028.
- [9] M. Gomez-Rodriguez, J. Leskovec, and B. Schölkopf. 2013. Modeling Information Propagation with Survival Theory. In *Proceedings of the 30th International Conference on Machine Learning (ICML 2013)*. 666–674.
- [10] M. Gomez-Rodriguez, J. Leskovec, and B. Schölkopf. 2013. Structure and Dynamics of Information Pathways in Online Media. In *Proceedings of the 6th ACM International Conference on Web Search and Data Mining (WSDM 2013)*. 23–32.
- [11] M. Gomez-Rodriguez and B. Schölkopf. 2012. Submodular Inference of Diffusion Networks from Multiple Trees. In *Proceedings of the 29th International Conference on Machine Learning (ICML 2012)*. 489–496.
- [12] K. Han, Y. Tian, Y. Zhang, L. Han, H. Huang, and Y. Gao. 2020. Statistical Estimation of Diffusion Network Topologies. In *Proceedings of the 36th IEEE International Conference on Data Engineering (ICDE 2020)*. 625–636.
- [13] S. He, H. Zha, and X. Ye. 2020. Network diffusions via neural mean-field dynamics. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2171–2183.
- [14] H. Huang, K. Han, B. Xu, and T. Gan. 2022. Reconstructing diffusion networks from incomplete data. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI 2022)*. 3085–3091.
- [15] H. Huang, K. Han, B. Xu, and T. Gan. 2023. Multi-aspect Diffusion Network Inference. In *Proceedings of the 32nd ACM Web Conference (WWW 2023)*. 82–90.
- [16] H. Huang, Q. Yan, L. Chen, Y. Gao, and C. S. Jensen. 2021. Statistical Inference of Diffusion Networks. *IEEE Transactions on Knowledge and Data Engineering* 33, 2 (2021), 742–753.
- [17] H. Huang, Q. Yan, T. Gan, D. Niu, W. Lu, and Y. Gao. 2019. Learning Diffusions without Timestamps. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence (AAAI 2019)*. 582–589.
- [18] H. Huang, Q. Yan, K. Han, T. Gan, J. Jiang, Q. Xu, and C. Yang. 2024. Learning Diffusions under Uncertainty. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*. 20430–20427.
- [19] D. Kalimeris, Y. Singer, K. Subbian, and U. Weinsberg. 2018. Learning Diffusion using Hyperparameters. In *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*. 2420–2428.
- [20] D. Kempe, J. Kleinberg, and É. Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2003)*. 137–146.
- [21] A. Lancichinetti, S. Fortunato, and F. Radicchi. 2008. Benchmark Graphs for Testing Community Detection Algorithms. *Physical Review E* 78, 4 (2008).
- [22] H. Li, C. Xia, T. Wang, S. Wen, C. Chen, and X. Yang. 2021. Capturing Dynamics of Information Diffusion in SNS: A Survey of Methodology and Techniques. *ACM Computing Surveys* 55, 1 (2021), 1–51.
- [23] S. Myers and J. Leskovec. 2010. On the Convexity of Latent Social Network Inference. In *Advances in Neural Information Processing Systems 23 (NIPS 2010)*. 1741–1749.
- [24] H. Narasimhan, D. C. Parkes, and Y. Singer. 2015. Learnability of Influence in Networks. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*. 3186–3194.
- [25] P. Netrapalli and S. Sanghavi. 2012. Learning the Graph of Epidemic Cascades. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS 2012)*. 211–222.
- [26] P. Papadimitriou, A. Dasdan, and H. Garcia-Molina. 2010. Web Graph Similarity for Anomaly Detection. *Journal of Internet Services and Applications* 1, 1 (2010), 19–30.
- [27] J. Pouget-Abadie and T. Horel. 2015. Inferring Graphs from Cascades: A Sparse Recovery Framework. In *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)*. 977–986.
- [28] M. Ramezani, A. Ahadina, E. Farhadi, and H. Rabiee. 2024. DANI: Fast diffusion aware network inference with preserving topological structure property. *Scientific Reports* 14, 1 (2024), 31053.
- [29] Y. Rong, Q. Zhu, and H. Cheng. 2016. A Model-Free Approach to Infer the Diffusion Network from Event Cascade. In *Proceedings of the 25th ACM International Conference on Information and Knowledge Management (CIKM 2016)*. 1653–1662.
- [30] E. Sefer and C. Kingsford. 2015. Convex Risk Minimization to Infer Networks from Probabilistic Diffusion Data at Multiple Scales. In *Proceedings of the 31st IEEE International Conference on Data Engineering (ICDE 2015)*. 663–674.
- [31] S. Wang, X. Hu, P. Yu, and Z. Li. 2014. MMRate: Inferring Multi-aspect Diffusion Networks with Multi-pattern Cascades. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2014)*. 1246–1255.
- [32] Y. Wang, D. Hou, C. Gao, X. Li, and Z. Wang. 2024. Inferring Information Diffusion Networks without Timestamps. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM 2024)*. 2453–2461.
- [33] Y. Wang, H. Wang, C. Gao, K. Fan, H. Cheng, Z. Shen, Z. Wang, and M. Perc. 2025. Learning influence probabilities in diffusion networks without timestamps. *Appl. Math. Comput.* 503 (2025), 129502.
- [34] Q. Yan, Y. Yang, K. Yin, T. Gan, and H. Huang. 2025. Diffusion Pattern Mining. *Knowledge and Information Systems* 67, 2 (2025), 1101–1129.
- [35] X. Zhang, Z. Zhang, W. Wang, D. Hou, J. Xu, X. Ye, and S. Li. 2021. Multiplex network reconstruction for the coupled spatial diffusion of infodemic and pandemic of COVID-19. *International Journal of Digital Earth* 14, 4 (2021), 401–423.
- [36] Y. Zhang and Q. Yang. 2021. A Survey on Multi-Task Learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 12 (2021), 5586–5609.

A Appendix

A.1 Proof of Lemma 1

Proof: Let

$$\hat{Q} = \prod_{s=1}^{t-1} p(D_t | G_t^{(T+1)}) \prod_{s=t+1}^k p(D_t | G_t^{(T)}), \quad (18)$$

then

$$\begin{aligned} & \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}) \\ &= f_t^{(T+1)}(G_t^{(T)}) \hat{Q}, \end{aligned} \quad (19)$$

$$\begin{aligned} & \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T+1)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}) \\ &= f_t^{(T+1)}(G_t^{(T+1)}) \hat{Q}. \end{aligned} \quad (20)$$

Since $G_t^{(T+1)} = \arg \max_{G_t} f_t^{(T+1)}(G_t)$, then

$$f_t^{(T+1)}(G_t^{(T)}) \leq f_t^{(T+1)}(G_t^{(T+1)}). \quad (21)$$

Thus, we have

$$f_t^{(T+1)}(G_t^{(T)}) \hat{Q} \leq f_t^{(T+1)}(G_t^{(T+1)}) \hat{Q}. \quad (22)$$

Combining with the equations Eq. (19) and Eq. (20) we know this lemma holds. ■

A.2 Proof of Lemma 2

Proof: Referring to the proof of Lemma 1, here we only need to prove that

$$f_t^{(T+1)}(G_t^{(T)}) \leq f_t^{(T+1)}(G_t^{(T+1)}), \quad (23)$$

and the equation holds iff $G_t^{(T+1)} = G_t^{(T)}$.

From Eqs. (12) and (13), we have

$$\begin{aligned} \ln f_t^{(T+1)}(G_t^{(T)}) &= \sum_{i=1}^n g_t^{(T+1)}(v_i, F_{t,i}^{(T)}), \\ \ln f_t^{(T+1)}(G_t^{(T+1)}) &= \sum_{i=1}^n g_t^{(T+1)}(v_i, F_{t,i}^{(T+1)}). \end{aligned} \quad (24)$$

Obviously, Algorithm 1 is a method for solving the optimization problem defined in Eq. (15). The candidate solution $F'_{t,i}$ is initialized as $F_{t,i}^{(T)}$ (line 2). After that, all the feasible solutions of Eq. (15) are taken into account. The candidate solution $F'_{t,i}$ would be updated only when the value of scoring function is strictly larger than that of current candidate solution $F'_{t,i}$ (lines 3-11). Given this, we have

$$g_t^{(T+1)}(v_i, F_{t,i}^{(T)}) \leq g_t^{(T+1)}(v_i, F_{t,i}^{(T+1)}), \quad (25)$$

the equality holds iff $F_{t,i}^{(T+1)} = F_{t,i}^{(T)}$. Summing the above inequalities over $i = 1, \dots, n$, we have

$$\sum_{i=1}^n g_t^{(T+1)}(v_i, F_{t,i}^{(T)}) \leq \sum_{i=1}^n g_t^{(T+1)}(v_i, F_{t,i}^{(T+1)}), \quad (26)$$

the equality holds iff $\forall i \leq n, F_{t,i}^{(T+1)} = F_{t,i}^{(T)}$, namely $G_t^{(T+1)} = G_t^{(T)}$. Combining with Eq. (24), we know Eq. (23) holds, and then this lemma holds. ■

A.3 Proof of Theorem 1

Proof: For t from 1 to k , using Lemma 2 we have

$$\begin{aligned} &\mathcal{L}(G_1^{(T)}, \dots, G_k^{(T)}) \\ &\leq \mathcal{L}(G_1^{(T+1)}, G_2^{(T)}, \dots, G_k^{(T)}) \\ &\dots \\ &\leq \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}) \\ &\leq \mathcal{L}(G_1^{(T+1)}, \dots, G_{t-1}^{(T+1)}, G_t^{(T+1)}, G_{t+1}^{(T)}, \dots, G_k^{(T)}) \\ &\dots \\ &\leq \mathcal{L}(G_1^{(T+1)}, \dots, G_k^{(T+1)}). \end{aligned} \quad (27)$$

There are k inequalities in (27), the conditions for the equality sign are $G_1^{(T+1)} = G_1^{(T)}, \dots, G_k^{(T+1)} = G_k^{(T)}$, respectively. Then, $\mathcal{L}(G_1^{(T)}, \dots, G_k^{(T)}) \leq \mathcal{L}(G_1^{(T+1)}, \dots, G_k^{(T+1)})$, and the equal sign holds if and only if $\forall 1 \leq t \leq k, G_t^{(T+1)} = G_t^{(T)}$. ■

A.4 Proof of Theorem 2

Proof: The while loop updates $G_1, \dots, G_k$ until the loop condition *change* = *true* not holds, which means for all $t \in \{1, \dots, k\}$ that $G'_t = G_t$, i.e., through a while loop $G_1, \dots, G_k$ not change. Let $G_1^{(0)}, \dots, G_k^{(0)}$ be the initial value of $G_1, \dots, G_k$ as line 1 in Algorithm 2. Let $G_1^{(T)}, \dots, G_k^{(T)}$ be the value of $G_1, \dots, G_k$ after T while loop. Then the while loop terminates when $\forall 1 \leq t \leq k, G_t^{(T+1)} = G_t^{(T)}$ holds. From Theorem 1 we know that

$$\begin{aligned} \mathcal{L}(G_1^{(0)}, \dots, G_k^{(0)}) &\leq \mathcal{L}(G_1^{(1)}, \dots, G_k^{(1)}) \\ &\leq \mathcal{L}(G_1^{(2)}, \dots, G_k^{(2)}) \leq \dots \end{aligned}$$

Assume the MIND algorithm does not terminate. Then for $T = 0, 1, 2, \dots$, the formula $\forall 1 \leq t \leq k, G_t^{(T+1)} = G_t^{(T)}$ does not hold, we have

$$\mathcal{L}(G_1^{(T)}, \dots, G_k^{(T)}) < \mathcal{L}(G_1^{(T+1)}, \dots, G_k^{(T+1)}). \quad (28)$$

Thus, we obtain a strictly increasing infinite sequence

$$\begin{aligned} \mathcal{L}(G_1^{(0)}, \dots, G_k^{(0)}) &< \mathcal{L}(G_1^{(1)}, \dots, G_k^{(1)}) \\ &< \mathcal{L}(G_1^{(2)}, \dots, G_k^{(2)}) < \dots \end{aligned} \quad (29)$$

Since the possible values of each G_t is discrete and finite (with at most $2^{n(n-1)}$ possible combinations), the values of $G_1, \dots, G_k$ are also discrete and finite, and we can have a conclusion that the possible value of $\mathcal{L}(G_1, \dots, G_k)$ is finite. If Eq. (29) is infinite, then $\mathcal{L}(G_1, \dots, G_k)$ has an infinite number of values, which is a contradiction with the earlier conclusion. Therefore, the MIND algorithm terminates in finite steps. ■